

Fenwick Tree

Day 1 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“updates and prefix sum queries”*
- ▶ *“a query for the value of $a[0] + a[1] + \dots + a[i-1]$ ”*

Programming interpretation:

- ▶ The array starts at zero and receives point increments.
- ▶ Every query asks for a prefix sum, not a closed interval ending at i .
- ▶ A Fenwick tree gives both operations logarithmic time.

Textbook connection: CP4 2.4.3 Fenwick (Binary Indexed) Tree

Core technique: Fenwick tree for dynamic prefix sums

- ▶ The array starts with all zeroes.
- ▶ '+ i v' adds v to one position.
- ▶ '? i' asks for the sum of positions before i .

- ▶ Naive prefix queries are too slow after many updates.
- ▶ Fenwick trees support point updates and prefix sums in logarithmic time.
- ▶ The main implementation detail is converting 0-based problem indices to 1-based tree indices.

- ▶ Build a Fenwick tree array $ft[1..N]$.
- ▶ $update(i, v)$: add v to $ft[i]$, then jump $i += l_{\text{one}}(i)$.
- ▶ $query(1, j)$: accumulate $ft[j]$, then jump $j -= l_{\text{one}}(j)$.
- ▶ For '+ i v', call $update(i + 1, v)$.
- ▶ For '? i', query positions 1 through i .

- ▶ $\text{lsone}(x) = x \& -x$ identifies the size of the chunk controlled by x .
- ▶ Update moves upward to every chunk containing the changed index.
- ▶ Query moves downward through disjoint chunks that cover the prefix.

- ▶ Each Fenwick cell stores the sum of a fixed suffix chunk ending at that index.
- ▶ The update visits exactly the chunks that include the changed position.
- ▶ The query decomposes a prefix into disjoint stored chunks, so their sum equals the prefix sum.

Complexity: $O(\log N)$ per update/query and $O(N)$ memory.

- ▶ Use integer arithmetic; sums may exceed small integer ranges in other languages.
- ▶ Kattis '?' is exclusive of i , which is why `query(1, i)` is correct after 1-based conversion.

Source lines: fenwick.py:34-47

```
class FTree:
    def __init__(self, f):
        self.n = len(f)
        self.ft = [0] * (self.n + 1)

        for i in range(1, self.n + 1):
            self.ft[i] += f[i - 1]

            parent = i + self.lstone(i)

            if parent <= self.n:
                self.ft[parent] += self.ft[i]
```

What this chunk does

- ▶ The Fenwick array has one extra cell because the structure is 1-based.
- ▶ The build loop pushes each value into the parent responsible for a larger range.
- ▶ For this problem the initial array is all zeros, but the constructor is general.

Source lines: fenwick.py:48-65

```
def lsone(self, s):  
    return s & (-s)  
  
def query(self, i, j):  
    if i > 1:  
        return self.query(1, j) - self.query(1, i - 1)  
  
    s = 0  
  
    while j > 0:  
        s += self.ft[j]  
        j -= self.lsone(j)  
  
    return s
```

What this chunk does

- ▶ lsone isolates the size of the range represented by the current tree node.
- ▶ A prefix query repeatedly jumps to the previous responsible node.
- ▶ A range query is handled as the difference of two prefix sums.

Source lines: fenwick.py:67-90

```
def update(self, i, v):
    while i <= self.n:
        self.ft[i] += v
        i += self.lsize(i)

def select(self, k):
    p = 1

    while (p * 2) <= self.n:
        p *= 2

    i = 0

    while p > 0:
        if i + p <= self.n and k > self.ft[i + p]:
            k -= self.ft[i + p]
            i += p

        p //= 2

    return i + 1
```

What this chunk does

- ▶ Updating position i means adding the delta to every Fenwick node whose range contains i .
- ▶ The index moves upward by $\text{lsize}(i)$.
- ▶ `select` is library functionality and is not needed by this Kattis problem.

Source lines: fenwick.py:93-112

```
def main():
    input = sys.stdin.buffer.readline

    n, q = map(int, input().split())

    ft = FTree([0] * n)

    output = []

    for _ in range(q):
        command = input().split()

        if command[0] == b"+":
            index = int(command[1])
            value = int(command[2])

            ft.update(index + 1, value)
```

What this chunk does

- ▶ The Kattis indices are 0-based, so updates use index + 1 inside the tree.
- ▶ A plus command increments one array position.
- ▶ The Fenwick update pushes the delta to every responsible tree node.

Source lines: fenwick.py:114-125

```
    else:
        index = int(command[1])

        output.append(str(ft.query(1, index)))

    sys.stdout.write("\n".join(output))

main()
```

What this chunk does

- ▶ The query '? i' maps directly to tree positions 1 through i.
- ▶ Each prefix sum is appended to the output buffer.
- ▶ All answers are printed once at the end.

Solution file: `practice_contests/day_1/fenwick.py`

Problem: <https://open.kattis.com/problems/fenwick>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.